

Lecture 04 - Structs and Methods

1

What is a struct?

Go's structs are typed collections of fields.
They're useful for grouping data together to form records.

```
type User struct {  
    ID int  
    Name string  
    Age int  
}
```

2

Struct fields

```
type User struct {  
    ID int  
    Name string  
    Age int  
}
```

```
user := User{  
    ID: 1,  
    Name: "Manas",  
    Age: 67,  
}
```

`user.Name` // accessing the field's value

3

Creating struct values

```
user := User{  
    Name: "Manas",  
}
```

`user.Age` ?

```
var user User
```

```
user := User{1, "Manas", 67, 22, 45, 91}
```

4

Structs use value semantics

```
type User struct {
    ID int
    Name string
    Age int
}
```

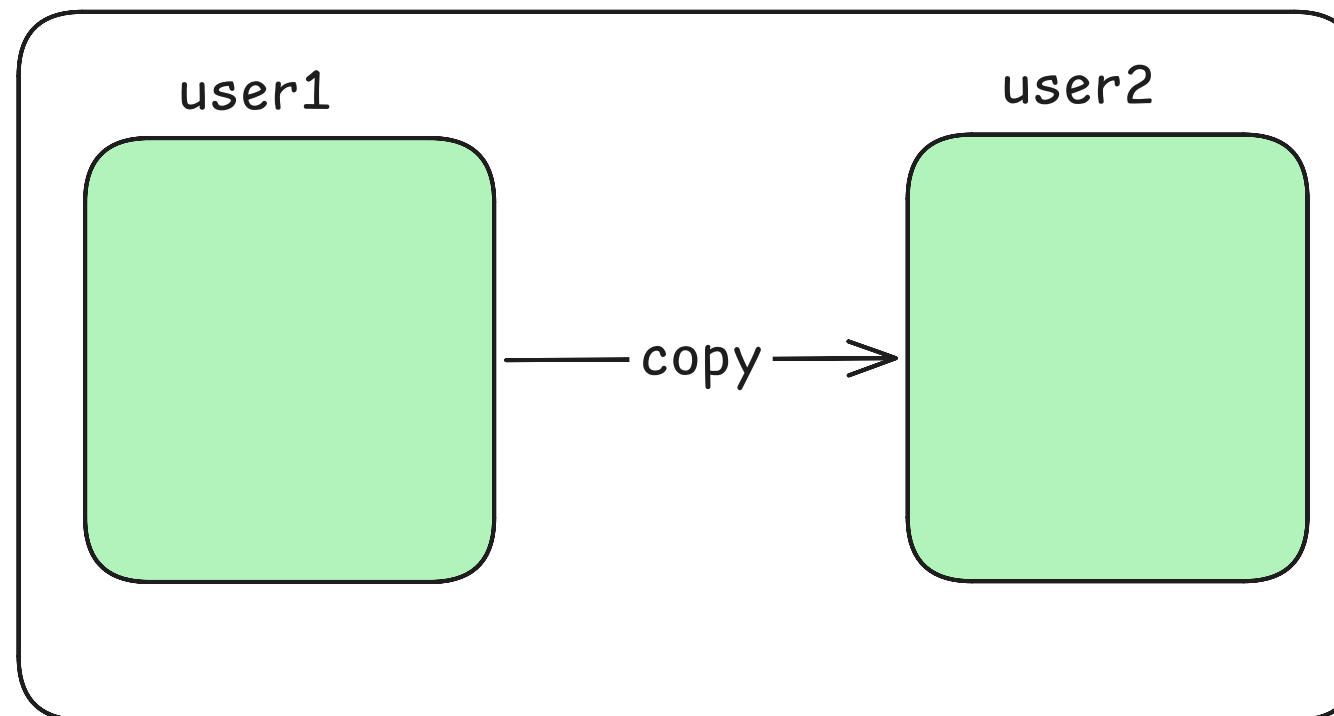
```
func main() {
    user1 := User{
        Name: "Manas",
    }

    user2 := user1

    user2.Name = "Baty"
    fmt.Println(user1.Name) // Manas
    fmt.Println(user2.Name) // Batyr

    rename(user1)
    fmt.Println(user1.Name) // Manas
}

// will use a copy of initial user
func rename(u User) {
    u.Name = "Baty"
}
```



Pointers

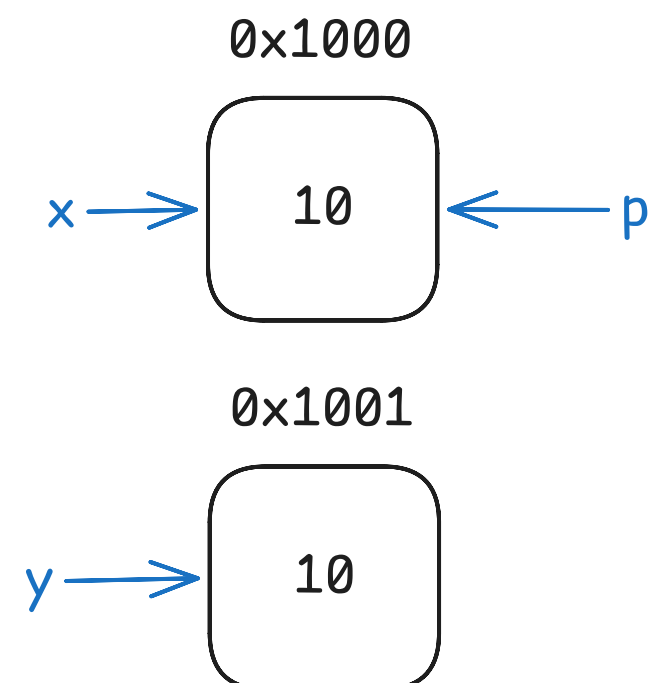
```
x := 10
```

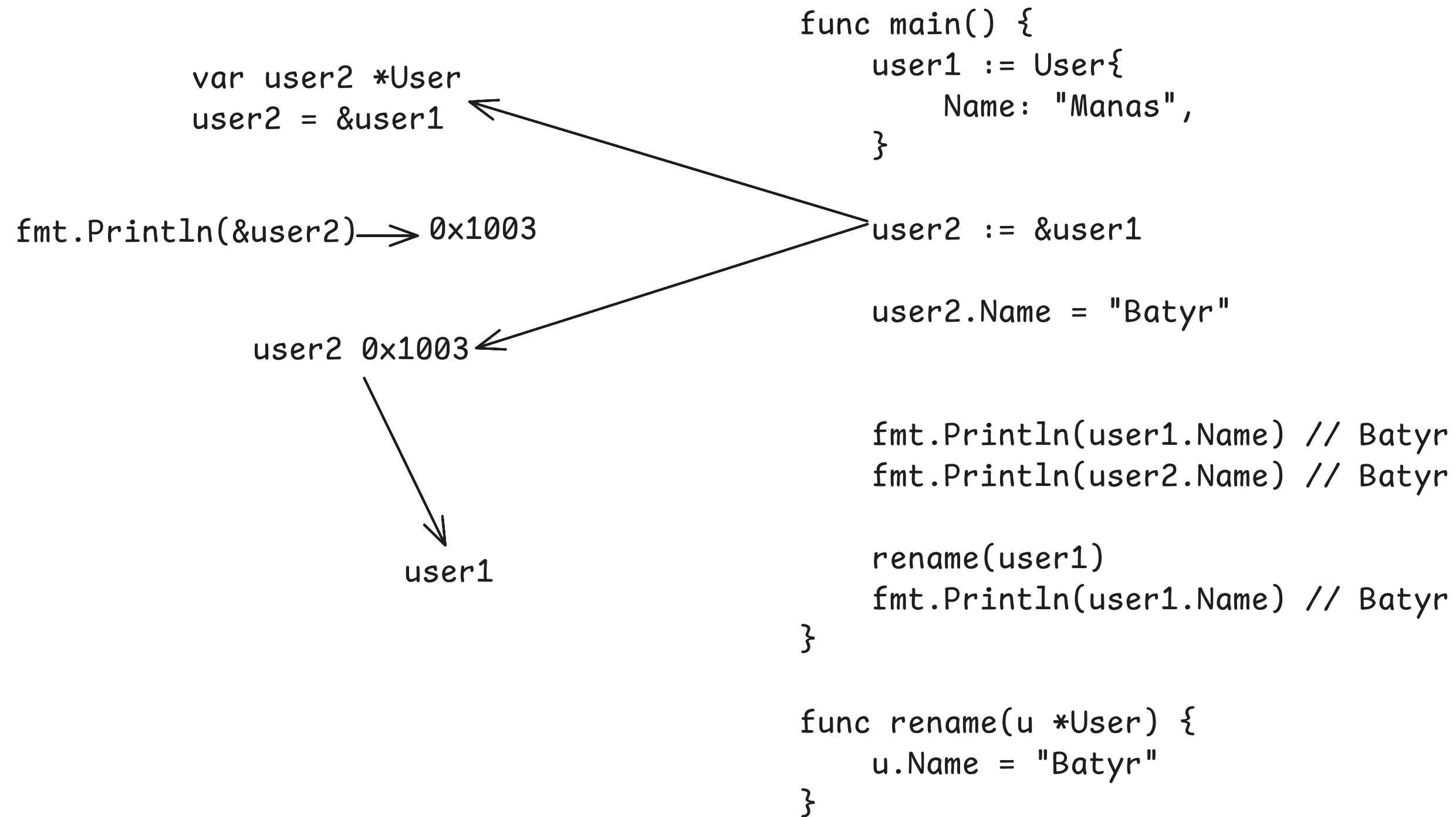
```
y := x
```

```
p := &x
```

give me
the
address of x

```
var x int
var p *int
```





user1 -> actual User
&user1 -> address of user1
user2 -> contains the address of user
*user2 -> actual User again
&user2 -> address of the pointer variable user2

5

What is a method?

Receiver

```
type Rectangle struct {
    Width float64
    Height float64
}
```

```
func main() {
    rectangle1 := Rectangle{
        Width: 10.0,
        Height: 5.0,
    }
```

```
    area := Area(rectangle1)
}
```

```
func Area(r Rectangle) float64 {
    return r.Width * r.Height
}
```

with receiver →

```
type Rectangle struct {
    Width float64
    Height float64
    Name string
}
```

```
func (r Rectangle) Area() float64 {
    return r.Width * r.Height
}
```

```
func (r *Rectangle) Rename(name string) {
    r.Name = name
}
```

```
func main() {
    rectangle1 := Rectangle{
        Width: 10.0,
        Height: 5.0,
        Name: "Yakubovich"
    }
```

```
    rectangle1.Area()
    rectangle1.Rename("Bob")
```

```
    fmt.Println(rectangle1.Name) // Bob
}
```

6

Receivers

Constructor functions

PHP constructor magic method (similar for majority OOP langs.)

```
class User
{
    public string $name;

    public function __construct()
    }

    $user = User();
```

In Golang, we rely on factory functions

```
type User struct {
    ID    int
    Name  string
    Age   int
}

// NewUser acts as the constructor for the User struct
func NewUser(id int, name string) *User {
    return &User{
        ID:    id,
        Name:  name,
        Role:  "Guest"
    }
}
```